01 A first-principles examination guide

Thirty focused pages based on the supplied 56-page annotated past-paper collection.

Data Structures and Algorithms

Learn the method. Draw the state. Verify the answer.

This guide starts with what a structure stores, then explains why each operation works. It covers the concept families in the supplied solutions for the 2023 final, 2024 re-examination and 2025 final. Problems are reorganised by concept; repeated questions share a method. It is a study companion, rather than a reproduction of every question.

Your aim is to be able to reproduce the reasoning under exam conditions. A score of nine depends on your exam's scale, marking and your practice; this guide cannot guarantee it. What you can control is writing correct algorithms, complete traces and clear checks.

A realistic one-night route: about seven hours of focused study

Read with a pen. After each worked example, cover the result and repeat the next step yourself. The last 45 minutes should be recall, not new reading. Take short breaks between blocks; keep some time for sleep.

Block	Pages	Task / approximate time
Foundations	2–6	Definitions, costs, arrays, lists, stacks: 65 minutes.
Expressions and queues	7–10	Reproduce stack tables and wraparound: 50 minutes.
Sorting	11–12	Write each pass and each merge: 30 minutes.
Trees	13–19	Follow every branch, then redraw without looking: 100 minutes.
Graphs	20–26	Trace queue, stack, distances and accepted edges: 100 minutes.
Coding and recall	27–30	Huffman, hashing and the closed-book check: 75 minutes.

Where to find a method quickly

Topic	Pages	Topic	Pages
Complexity; recursion	2–3	BFS; DFS	21–22
Linked lists; stacks	4–6	Dijkstra; directed routes	23–24
Infix / postfix / prefix	7–9	Prim; Kruskal	25–26
Queues; sorting	10–12	Huffman	27–28
Tree terms; BST insertion	13–17	Hashing; heaps	2, 29
Tree reconstruction; deletion	18–19	Self-test; coverage map	30

Conventions used throughout

Arrays start at index 0. Stack contents are written bottom to top, with the top at the right. A graph arrow is directed; a plain edge is undirected. Neighbours are alphabetical or numerical unless stated. Tree height counts edges; the root has depth 0 and level 1. Missing tree children are empty links. Tree snapshots are read from the top down; a thicker circle marks the new or changed key.

Algorithms use **Step 0: Start**, numbered actions, and a final **End**. Assignments such as $d[v] \leftarrow 5$ mean "store 5 in that variable." NULL means "no node." Each worked trace has a check you can use independently.

02 Foundations, costs and choosing a structure

Source links: 2023 Q1(a), Q2(a); 2024 Q2(a), Q7(c); 2025 Q1(d), Q2(a).

A **data structure** organises values and their relationships. An **algorithm** is a finite procedure for a task. An **abstract data type** describes allowed operations (a stack supports push and pop); its implementation may be an array or linked nodes. Linear structures form a sequence. Trees represent a hierarchy. General graphs represent arbitrary connections.

Count growth, not seconds

Let n be the number of items; V, E count graph vertices and edges. A constant-time operation does a bounded amount of work regardless of n. Reading all n items takes linear time. Repeatedly halving a range takes logarithmic time: $1024 \to 512 \to \cdots \to 1$ needs 10 halvings.

Notation	Meaning in plain language	Example
$O(g(n))$	An eventual upper bound, up to a constant factor.	$3n^2 + n = O(n^2)$
$\Omega(g(n))$	An eventual lower bound, up to a positive constant.	$3n^2 + n = \Omega(n^2)$
$\Theta(g(n))$	Matching upper and lower growth bounds.	$3n^2 + n = \Theta(n^2)$
$o(g(n))$	Strictly smaller growth; ratio tends to zero.	$n = o(n^2)$
$\omega(g(n))$	Strictly larger growth; ratio grows without bound.	$n^2 = \omega(n)$

Formally, $f = O(g)$ if some $c > 0, n_0$ give $0 \le f(n) \le cg(n)$ for all $n \ge n_0$. Replace $\le$ with $\ge$ for Ω; require both bounds for Θ. Big-O itself does not mean "worst case": specify which input case you analyse.

For `s=0; for each A[i]: s=s+A[i]`, the body executes n times. Thus $T(n) = a + bn = \Theta(n)$. Multiplying all array entries has the same growth. Two full nested n-loops cost $\Theta(n^2)$; two consecutive n-loops cost $\Theta(n)$, not $\Theta(n^2)$. A triangular loop costs $1 + 2 + \cdots + n = n(n+1)/2 = \Theta(n^2)$.

Space: auxiliary space excludes the input. Summing uses $O(1)$ auxiliary space, even though its input array occupies $O(n)$. Recursive calls use memory too. Standard array merge sort uses $O(n)$ auxiliary space.

Need	Choose	Reason / qualification
Read item at index i	Array	$O(1)$ access; middle insertion may shift $O(n)$ items.
Edit after a known node	Linked list	$O(1)$ relinking; finding the node can cost $O(n)$.
Undo / nested work	Stack	Last in, first out (LIFO).
Serve arrivals / buffer	Queue / circular queue	First in, first out (FIFO); circle reuses array cells.
Find ticket ID	Hash table	Expected $O(1)$ lookup; $O(n)$ worst case.
Smallest job next	Min-heap	Minimum at root; peek $O(1)$, insert / remove $O(\log n)$.

A **binary min-heap** is a complete binary tree: levels fill left to right, and a parent is no larger than either child. It is not a BST: left and right subtrees need not be ordered against each other. For array index i, children are $2i + 1, 2i + 2$. Insert at the end and swap upward while smaller than the parent; remove-min by moving the last item to the root and swapping downward with the smaller child. These paths have $O(\log n)$ height.

Remember. Say what is counted, name the input size, and include the implementation assumption.

03 Arrays, binary search and recursion

Source links: 2023 Q2(a), Q6(b); 2024 Q1(a), Q2(a).

An array stores entries in consecutive indexed slots. Its current size n can be smaller than its capacity C. To insert, first make a free slot; moving rightmost entries first prevents overwriting values you still need.

Algorithm: Insert x at array index p

Step 0. Start.

Step 1. If $n = C$, report overflow and go to End. If $p < 0$ or $p > n$, report invalid position and go to End.

Step 2. For $i = n-1, n-2, \ldots, p$, assign $A[i+1] \leftarrow A[i]$.

Step 3. Assign $A[p] \leftarrow x$; set $n \leftarrow n+1$; return the new size.

Step 4. End.

Example: insert 25 at index 2 of $[10, 20, 30, 40]$, capacity 5.

$$[10, 20, 30, 40, _] \;\rightarrow\; [10, 20, 30, 40, 40] \;\rightarrow\; [10, 20, 30, 30, 40] \;\rightarrow\; [10, 20, 25, 30, 40].$$

The temporary duplicates are copies, not extra logical items. Cost is $O(n)$ worst case, $O(1)$ when appending with space. To delete index p, shift $A[p+1], \ldots, A[n-1]$ one slot left and reduce n.

Binary search: the sorted order lets you discard half

If the middle value is smaller than the target, every value to its left is also too small. The input must be sorted; indexing should be efficient.

Algorithm: Binary search for target x in ascending array A

Step 0. Start.

Step 1. Set $low \leftarrow 0$, $high \leftarrow n-1$.

Step 2. If $low > high$, return "not found and go to End. Otherwise set $mid \leftarrow low + \lfloor (high - low)/2 \rfloor$.

Step 3. If $A[mid] = x$, return mid and go to End.

Step 4. If $A[mid] < x$, set $low \leftarrow mid + 1$; otherwise set $high \leftarrow mid - 1$. Go to Step 2.

Step 5. End.

Step	*low*	*high*	*mid*	Middle value / decision
1	0	10	5	76: target 98 is larger; low becomes 6.
2	6	10	8	93: low becomes 9.
3	9	10	9	97: low becomes 10.
4	10	10	10	98: found at index 10, the 11th position.

The searched array is $[4, 21, 34, 40, 65, 76, 80, 92, 93, 97, 98]$. Use $mid \pm 1$, or the same midpoint can repeat forever. Iterative search uses $O(\log n)$ worst-case time and $O(1)$ auxiliary space.

Recursion: pause a problem while solving a smaller one

Algorithm: Factorial $F(n)$ for a nonnegative integer

Step 0. Start.

Step 1. If $n < 0$ or is not an integer, report invalid input and go to End.

Step 2. If $n = 0$ or $n = 1$, return 1 and go to End.

Step 3. Return $n \times F(n-1)$; the recursive call follows these same steps.

Step 4. End.

For $F(4)$, calls descend $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$; results return $1 \rightarrow 2 \rightarrow 6 \rightarrow 24$. A **base case** stops recursion; the input must progress toward it. Time and call-stack space are both $\Theta(n)$ under the usual unit-cost arithmetic model.

04 Linked lists: pictures before pointer changes

Source links: 2023 Q1(c), Q6(a); 2024 Q1(b); 2025 Q3(a), Q6(b).

A singly linked node contains `data` and `next`. `head` stores the address of the first node, not its value. `NULL` ends a noncircular list. Nodes can occupy unrelated memory locations: following `next`, not moving to the next physical address, gives the logical order.

Address	Data	Next address	Meaning
100	Adil	420	head points to 100
420	Adnan	260	follow the saved address 260
260	Firoz	NULL	final node

head $\longrightarrow$ [Adil | 420] $\longrightarrow$ [Adnan | 260] $\longrightarrow$ [Firoz | NULL].

The addresses here are illustrative; bench or roll numbers are record fields, not automatically addresses.

Past-paper names: sorted by name, not by bench

Occupied benches are 1 Kavita, 3 Diksha, 4 Adil, 5 Adnan, 7 Lara, 9 Sahil, 10 Firoz. Empty benches create no nodes.

Stage	Logical chain (each arrow is a next pointer)
Initial sort	head $\to$ Adil $\to$ Adnan $\to$ Diksha $\to$ Firoz $\to$ Kavita $\to$ Lara $\to$ Sahil $\to$ NULL
Remove bench 3	Adnan.next now points to Firoz; Diksha is bypassed. Other links stay as before.
Remove bench 7	Kavita.next now points to Sahil; Lara is bypassed.
Insert Roma, bench 8	First Roma.next $\leftarrow$ Sahil; then Kavita.next $\leftarrow$ Roma.
Final	head $\to$ Adil $\to$ Adnan $\to$ Firoz $\to$ Kavita $\to$ Roma $\to$ Sahil $\to$ NULL

Past-paper marks: the comparison field can be different

Store each record as $[\,roll \mid marks \mid next\,]$. With $(roll, marks) = (1,50), (3,60), (4,20), (5,30), (7,40), (9,90), (10,35)$:

$$\begin{aligned}
\text{Initial: } & (4,20) \to (5,30) \to (10,35) \to (7,40) \to (1,50) \to (3,60) \to (9,90),\\
\text{Delete roll 3: } & (4,20) \to (5,30) \to (10,35) \to (7,40) \to (1,50) \to (9,90),\\
\text{Delete roll 7: } & (4,20) \to (5,30) \to (10,35) \to (1,50) \to (9,90),\\
\text{Insert } (8,59) : & (4,20) \to (5,30) \to (10,35) \to (1,50) \to (8,59) \to (9,90).
\end{aligned}$$

Every displayed chain begins at head and ends at NULL. Roll 8 goes between marks 50 and 90, because 59 is the sort key. "Delete roll 3" means find that record; it does not mean delete the third node.

The variants you must recognise

Singly: $head \to A \to B \to NULL$. One next pointer; forward navigation.

Doubly: $NULL \leftarrow A \rightleftarrows B \to NULL$. Each node has prev and next; maintain both directions. Given a middle node, deletion can use its prev pointer directly.

Circular singly: the tail.next points back to head. Stop traversal on returning to head, rather than waiting for NULL. A circular doubly linked list also links head.prev to the tail.

Check your answer. Count only occupied records; keep the chosen sort order; preserve access to the rest of the chain before replacing any link.

05 Linked-list algorithms you can reproduce

Source links: 2023 Q6(a); 2024 Q4(a), Q6(a); 2025 Q3(a), Q6(b).

The repeated rule is **save the successor, then redirect the predecessor**. An actual node pointer and a value to search for are different inputs.

Algorithm: Insert value x after known node p

Step 0. Start.

Step 1. If $p =$ NULL, report invalid node and go to End. Allocate node q; if allocation fails, report failure and go to End.

Step 2. Set $q.data \leftarrow x$ and $q.next \leftarrow p.next$. This preserves the old successor.

Step 3. Set $p.next \leftarrow q$. Return head unchanged.

Step 4. End.

For $10 \rightarrow 20 \rightarrow 30$, insert 25 after 20: first $q = (25, next \rightarrow 30)$, while 20 still points to 30; then point 20 to q. Result: $10 \rightarrow 20 \rightarrow 25 \rightarrow 30$. Reversing those assignments can lose 30 or create a self-loop. If given only value 20, first walk from head until it is found or NULL; this search costs $O(n)$.

Algorithm: Append x to a singly linked list with only head stored

Step 0. Start.

Step 1. Allocate q; on failure report failure and go to End. Set $q.data \leftarrow x$, $q.next \leftarrow$ NULL.

Step 2. If head is NULL, return q as the new head and go to End.

Step 3. Set $p \leftarrow head$. While $p.next \neq$ NULL, set $p \leftarrow p.next$.

Step 4. Set $p.next \leftarrow q$; return head.

Step 5. End.

Algorithm: Delete the first node

Step 0. Start.

Step 1. If head is NULL, report underflow and go to End.

Step 2. Save $old \leftarrow head$ and $x \leftarrow old.data$; set $head \leftarrow old.next$.

Step 3. Release old if memory is manually managed; return the new head and x.

Step 4. End.

Algorithm: Delete the first node whose value equals key

Step 0. Start.

Step 1. Set $prev \leftarrow$ NULL and $cur \leftarrow head$.

Step 2. While $cur \neq$ NULL and $cur.data \neq key$, set $prev \leftarrow cur$, then $cur \leftarrow cur.next$.

Step 3. If $cur =$ NULL, report *not found*, return head unchanged, and go to End.

Step 4. If $prev =$ NULL, set $head \leftarrow cur.next$; otherwise set $prev.next \leftarrow cur.next$.

Step 5. Release cur; return head and *deleted*.

Step 6. End.

Deleting 30 from $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$ stops with $prev = 20, cur = 30$. Link 20 directly to 40, then release 30. For key 99, the search reaches NULL and changes no link. Always return or otherwise update head when deleting its node.

A linked stack is just a list with its top at the head

Algorithm: Linked-stack push and pop

Step 0. Start.

Step 1. For PUSH(x): allocate q; if it fails, report overflow and go to End. Set $q.data \leftarrow x$, $q.next \leftarrow top$, then $top \leftarrow q$; go to End.

Step 2. For POP: if $top =$ NULL, report underflow and go to End.

Step 3. Save $q \leftarrow top$, $x \leftarrow q.data$; set $top \leftarrow q.next$; release q and return x.

Step 4. End.

Known-node insertion, head deletion and linked push/pop cost $O(1)$. Search and head-only append cost $O(n)$; keeping a tail makes append $O(1)$ if all updates maintain it. Linked-stack overflow means allocation failed (or an imposed limit), not "top equals capacity minus one."

06 Stacks and balanced parentheses

Source links: 2025 Q1(a), Q2(b); expressions also use this same LIFO idea.

A stack is a pile with access at one end: the **top**. PUSH adds, POP removes and returns, PEEK returns without removing. LIFO means the most recent unfinished item is handled first. In an array stack of capacity C, initialise $top = -1$; occupied slots are 0 through top.

Algorithm: Array-stack operations

Step 0. Start.

Step 1. For isEmpty: return whether $top = -1$ and go to End.

Step 2. For PUSH(x): if $top = C - 1$, report overflow and go to End. Otherwise set $top \leftarrow top + 1$, $A[top] \leftarrow x$ and go to End.

Step 3. For POP or PEEK: if $top = -1$, report underflow and go to End.

Step 4. Save $x \leftarrow A[top]$. For POP only, set $top \leftarrow top - 1$. Return x.

Step 5. End.

The operations are alternatives, not a requirement to run all four. Each takes $O(1)$ time.

Operation, capacity 5	Stack, bottom → top	Returned value / effect
Start	[]	top = −1
Push 10; Push 20; Push 30	[10, 20, 30]	After each push: [10], [10, 20], [10, 20, 30].
Peek()	[10, 20, 30]	30; no change
Pop()	[10, 20]	30; top moves down
Push 40	[10, 20, 40]	top = 2
Push 50	[10, 20, 40, 50]	top = 3
Push 60	[10, 20, 40, 50, 60]	top = 4; full
isEmpty()	[10, 20, 40, 50, 60]	false
Push 70	[10, 20, 40, 50, 60]	Overflow; failed push changes nothing.

Why brackets need a stack

In [($a + b$)], the most recently opened bracket must close first. A counter alone cannot reject ([)], because counts match but nesting is wrong.

Algorithm: Check matching parentheses, square brackets and braces

Step 0. Start.

Step 1. Create an empty stack S. Scan the input from left to right.

Step 2. For an opening bracket, push it. For other non-bracket symbols, do nothing.

Step 3. For a closing bracket: if S is empty, return false and go to End; otherwise pop the opening bracket. If its type does not match, return false and go to End.

Step 4. After every symbol is processed, return true exactly when S is empty.

Step 5. End.

Input fragment: $(a - c) * (b + c)$	Action	Stack
(	Push	(
$a - c$	Ignore	(
)	Pop matching (	empty
$*$ then (	Ignore multiplication; push opening	(
$b + c$ then)	Ignore symbols; pop matching opening	empty
End	Empty stack means balanced	empty

For ([)], push (then [; the next) fails because the top is [. For ((), one opening remains at the end, so reject. Time $O(n)$; worst-case auxiliary space $O(n)$. Balanced brackets do not by themselves prove that the entire expression is valid.

07 Infix to postfix: every stack decision

Source links: 2023 Q1(b); 2024 Q3(b); 2025 Q5(a).

Infix: $A + B$; **prefix / Polish:** `+ A B`; **postfix / Reverse Polish:** `A B +`. Postfix places an operator after both inputs. A token is one complete operand or operator: the number 12 is one token, not 1 followed by 2.

Precedence: $\wedge$ (power) first, then $*, /$, then $+, -$. Power is right-associative: $A \wedge B \wedge C = A \wedge (B \wedge C)$. The other listed binary operators are left-associative: $A/B * C = (A/B) * C$. Parentheses override precedence.

Algorithm: Convert valid binary-operator infix to postfix

Step 0. Start.

Step 1. Create empty operator stack S and empty output. Read tokens left to right.

Step 2. For an operand, append it to output. For an opening parenthesis, push it.

Step 3. For a closing parenthesis, pop operators to output until the matching opening parenthesis is on top; discard that opening. A missing match is an error.

Step 4. For operator t, pop to output while the top is an operator of higher precedence, or equal precedence when t is left-associative. Then push t. Never pop across an opening parenthesis.

Step 5. Repeat Steps 2–4 for all tokens. At the end, pop remaining operators to output. A remaining parenthesis is an error. Return output.

Step 6. End.

Exact paper expression: $(A - 2 * (B + C)/D * E) + F$

The middle chain is $((2 * (B + C))/D) * E$. Equal-precedence multiplication and division must be processed left to right. Every row below records the state *after* that token.

Token	Action	Stack, bottom → top	Postfix output
(	Push	(	empty
A	Output	(	`A`
$-$	Push	($-$	`A`
2	Output	($-$	`A 2`
$*$	Higher precedence: push	($-$ $*$	`A 2`
(	Push	($-$ $*$ (	`A 2`
B	Output	($-$ $*$ (	`A 2 B`
$+$	Push	($-$ $*$ ($+$	`A 2 B`
C	Output	($-$ $*$ ($+$	`A 2 B C`
)	Pop $+$; discard (	($-$ $*$	`A 2 B C +`
/	Pop equal $*$, then push /	($-$ /	`A 2 B C + *`
D	Output	($-$ /	`A 2 B C + * D`
$*$	Pop equal /, then push $*$	($-$ $*$	`A 2 B C + * D /`
E	Output	($-$ $*$	`A 2 B C + * D / E`
)	Pop $*$, $-$; discard (	empty	`A 2 B C + * D / E * -`
$+$	Push	$+$	`A 2 B C + * D / E * -`
F	Output	$+$	`A 2 B C + * D / E * - F`
End	Pop $+$	empty	`A 2 B C + * D / E * - F +`

Remember. Output operands immediately; postpone an operator only while it is waiting for its right input. At equal precedence, pop for incoming left-associative operators, but not for incoming power.

08 Postfix to infix, evaluation and prefix

Source links: 2024 Q2(a)(ii); 2023 Q1(b). Postfix-to-infix is added for transfer practice.

Both conversion and evaluation use the same rule: **first pop = right operand; second pop = left operand**. At an operator, its two complete inputs have already been built. Preserve that order for subtraction and division.

Algorithm: Convert postfix to fully parenthesised infix

Step 0. Start.

Step 1. Create an empty stack; scan tokens left to right. Push the text of each operand.

Step 2. At a binary operator *op*, require two stack items; otherwise report invalid input and go to End.

Step 3. Pop R, then L. Form (L *op* R) and push that complete string. Repeat for every token.

Step 4. At the end, require exactly one item; return it. Otherwise report invalid input.

Step 5. End.

Algorithm: Evaluate postfix numerically

Step 0. Start.

Step 1. Create an empty numeric stack; scan left to right. Push each number.

Step 2. At an operator, require two values; pop R then L. Calculate L *op* R and push the result. Reject undefined operations, such as division by zero.

Step 3. After all tokens, require exactly one value and return it; otherwise reject the expression.

Step 4. End.

Two complete traces: stack top is at the right

The left trace builds text; the right trace computes numbers. Every token has a row. Semicolons separate complete expressions on the left stack.

Convert: `A B + C * D E - /`

Token	Stack after token
A	`A`
B	`A ; B`
+	`(A+B)`
C	`(A+B) ; C`
*	`((A+B)*C)`
D	`((A+B)*C) ; D`
E	`((A+B)*C) ; D ; E`
-	`((A+B)*C) ; (D-E)`
/	`(((A+B)*C)/(D-E))`

Evaluate: `9 3 4 * 8 + 4 / -`

Token	Stack after token	Operation
9	[9]	push
3	[9, 3]	push
4	[9, 3, 4]	push
*	[9, 12]	$3 * 4 = 12$
8	[9, 12, 8]	push
+	[9, 20]	$12 + 8 = 20$
4	[9, 20, 4]	push
/	[9, 5]	$20/4 = 5$
-	[4]	$9 - 5 = 4$

At the left trace's minus, $R = E, L = D$, so build $(D-E)$. At the final division, $R = (D-E)$ and $L = ((A+B)*C)$, giving $((A + B) * C)/(D - E)$, not its reciprocal. In the numeric trace, the final minus pops 5 first, then 9, and computes $9 - 5 = 4$.

Validity check: each binary operator needs two complete inputs and reduces stack size by one. Finishing with zero items or several items is invalid. Operators must not be applied before both inputs exist.

Prefix without a reverse-and-guess shortcut

Algorithm: Infix to prefix through an expression tree

Step 0. Start.

Step 1. Convert infix to postfix by the numbered steps on page 7.

Step 2. Build an expression tree by the numbered steps on page 9; preserve left and right operands.

Step 3. Output preorder (root–left–right); return that prefix sequence.

Step 4. End.

For $(A + B) + C - (D - E) \wedge F$, postfix is `A B + C + D E - F ^ -`; prefix is `- + + A B C ^ - D E F`. The final subtraction is the root. For postfix-to-prefix directly, use the conversion algorithm above but form `op L R` at Step 3. Tree methods use $O(n)$ stack operations and avoid repeated copying of long strings.

09 Expression trees: build each operation

Source links: 2023 Q7(a); 2024 Q7(a). Each displayed step draws the newly completed subtree.

Leaves are operands; internal nodes are operators. The tree records grouping rather than alphabetical or numerical order. It is *not* a BST. Use postfix to know when two input subtrees are ready.

Algorithm: Build an expression tree from valid postfix

Step 0. Start.

Step 1. Create an empty stack of tree roots. For each operand, push a new leaf.

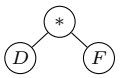
Step 2. At an operator, require two roots. Pop right subtree R, then left subtree L; make a new operator root with children L, R; push it.

Step 3. Process all tokens. Exactly one root must remain; return it. Read preorder for prefix, postorder for postfix, or parenthesised inorder for infix.

Step 4. End.

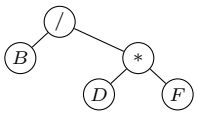
Example 1: $A + (B/(D * F) - H) * T$. Postfix: `A B D F * / H - T * +`. Single-letter pushes create leaves; the five operator encounters create these trees. Existing subtrees are preserved inside the next drawing.

1. Merge D, F



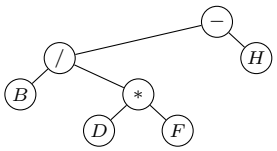
$D * F$: operator above its two inputs.

2. Merge B, product



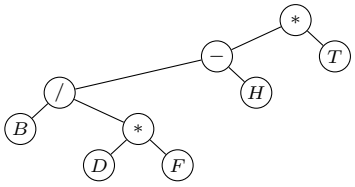
$B/(D * F)$: denominator on the right.

3. Subtract H



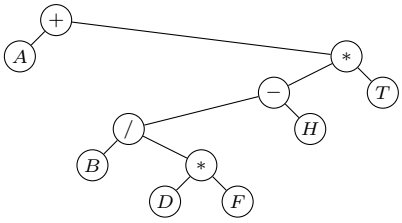
$(B/(D * F)) - H$.

4. Multiply by T



Previous expression is the left input.

5. Add A



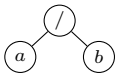
A goes left; the completed product goes right.

Prefix: `+ A * - / B * D F H T`. **Postfix:** `A B D F * / H - T * +`.

Example 2: read the paper's given operator tree

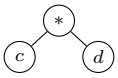
Its expression is $((a/b) + (c * d)) \wedge ((f\%g)/(h - i))$. Here % is remainder. The following seven merges build it from scratch; they also show how to translate every branch.

1. Divide



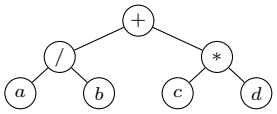
Left operand a, right operand b.

2. Multiply



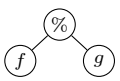
Left operand c, right operand d.

3. Add



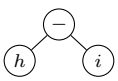
Combine steps 1 and 2.

4. Remainder



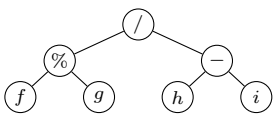
Compute $f\%g$.

5. Subtract



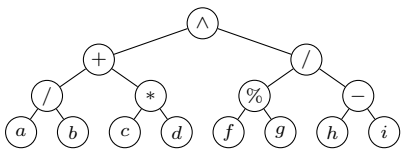
Compute $h - i$.

6. Divide



Step 4 divided by step 5.

7. Power



Step 3 is base; step 6 is exponent.

10 Queues: linear storage and circular reuse

Source links: 2023 Q2(b); 2024 Q2(b), Q7(b); 2025 Q7(a).

A queue is FIFO: insertion joins the rear; removal takes the front. It models waiting customers, printer jobs and BFS work. Unlike a stack, it removes the oldest waiting item.

Linear array queue: a simple implementation

Use $front = rear = -1$ for empty. Choose the requested operation below.

Algorithm: Linear array queue

Step 0. Start.

Step 1. For ENQUEUE(x): if $rear = C - 1$, report overflow and go to End. If empty, set $front = 0$. Increase rear by 1; store $Q[rear] = x$; go to End.

Step 2. For DEQUEUE: if $front = -1$, report underflow and go to End. Save $x = Q[front]$. If $front = rear$, reset both to -1; otherwise increase front by 1. Return x.

Step 3. End.

After filling capacity 5 with A–E and removing A,B, rear remains 4. A simple linear queue cannot append despite unused slots 0,1. A circular queue reuses them without shifting items.

Circular queue: the array is logically a ring

The next index is $(i + 1) \bmod C$. Modulo is the remainder: $(4 + 1) \bmod 5 = 0$. Use the same empty sentinel; for $C > 0$:

$$\boxed{\text{Empty: } front = -1} \qquad \boxed{\text{Full: } (rear + 1) \bmod C = front}.$$

These tests belong to this representation. Do not mix them with a queue implementation that uses a separate item count or deliberately leaves one slot unused.

Algorithm: Circular ENQUEUE or DEQUEUE

Step 0. Start.

Step 1. For DEQUEUE, go to Step 4. For ENQUEUE, if $(rear + 1) \bmod C = front$, report overflow and go to End.

Step 2. If $front = -1$, set $front = rear = 0$; otherwise set $rear \leftarrow (rear + 1) \bmod C$.

Step 3. Set $Q[rear] \leftarrow x$ and go to End.

Step 4. If $front = -1$, report underflow and go to End.

Step 5. Save $x \leftarrow Q[front]$. If $front = rear$, set both to -1; otherwise set $front \leftarrow (front + 1) \bmod C$.

Step 6. Return x. Remaining old bits in removed slots are not logical queue items.

Step 7. End.

Algorithm: Display a circular queue in FIFO order

Step 0. Start.

Step 1. If $front = -1$, report empty and go to End. Set $i \leftarrow front$.

Step 2. Print $Q[i]$. If $i = rear$, go to End. Otherwise set $i \leftarrow (i + 1) \bmod C$ and repeat this step.

Step 3. End.

Operation, capacity 5	$front, rear$	Physical array, indices 0–4	Logical FIFO order
Fill with A–E	0,4	A B C D E	A B C D E
Dequeue A	1,4	– B C D E	B C D E
Dequeue B	2,4	– – C D E	C D E
Enqueue F	2,0	F – C D E	C D E F
Enqueue G	2,1	F G C D E	C D E F G
Enqueue H	2,1	unchanged	Overflow: $(1 + 1) \bmod 5 = 2$.

Keep dequeuing: C, D, E, F, G leave in that order. Removing the final G resets $front, rear$ to $-1, -1$; one more dequeue underflows. Enqueue/dequeue cost $O(1)$, display $O(k)$ for k items. Applications include bounded streaming buffers and cyclic task queues used for round-robin service.

Remember. Draw physical slots for wraparound, but write logical FIFO order separately.

11 Selection sort and insertion sort

Source links: 2023 Q3(a); 2024 Q7(c); 2025 Q1(b).

Sorting means arranging values, here ascending, while preserving all occurrences. A **pass** is one outer-loop iteration. Do not write only the final sorted array: the passes show which algorithm you used.

Selection: fix the next smallest value permanently

Algorithm: Selection sort

Step 0. Start.

Step 1. For each position $i = 0, 1, \ldots, n-2$, start with $m \leftarrow i$.

Step 2. For $j = i+1, \ldots, n-1$, if $A[j] < A[m]$, set $m \leftarrow j$.

Step 3. Swap $A[i]$ and $A[m]$; the prefix through i is now final. Repeat Steps 1–3 for each i. Return A.

Step 4. End.

Pass	Minimum and swap	Array after pass
0	Original	45,32,12,23,30,6,15,40,17
1	6 swaps with 45	6,32,12,23,30,45,15,40,17
2	12 swaps with 32	6,12,32,23,30,45,15,40,17
3	15 swaps with 32	6,12,15,23,30,45,32,40,17
4	17 swaps with 23	6,12,15,17,30,45,32,40,23
5	23 swaps with 30	6,12,15,17,23,45,32,40,30
6	30 swaps with 45	6,12,15,17,23,30,32,40,45
7	32 already in place	6,12,15,17,23,30,32,40,45
8	40 already in place	6,12,15,17,23,30,32,40,45

Each pass scans the entire remaining suffix. Comparisons total $8+7+\cdots+1 = 36$ here; generally $n(n-1)/2 = \Theta(n^2)$, even when input is sorted. Auxiliary space is $O(1)$. Standard swapping selection sort is not stable: it can reverse equal-key records.

Insertion: place the next card into an already sorted hand

Algorithm: Insertion sort

Step 0. Start.

Step 1. For $i = 1, \ldots, n-1$, save $key \leftarrow A[i]$ and set $j \leftarrow i-1$.

Step 2. While $j \geq 0$ and $A[j] > key$, set $A[j+1] \leftarrow A[j]$ and decrease j by 1.

Step 3. Set $A[j+1] \leftarrow key$; repeat for the next i. Return A.

Step 4. End.

For $[4, 3, 1, 5, 2]$: insert 3 by shifting 4; insert 1 by shifting 4 then 3; 5 already fits; insert 2 by shifting 5,4,3. Save key before shifting, because its original array slot can be overwritten.

Pass	Selection sort	Insertion sort
Original	4,3,1,5,2	4,3,1,5,2
1	1,3,4,5,2	3,4,1,5,2
2	1,2,4,5,3	1,3,4,5,2
3	1,2,3,5,4	1,3,4,5,2
4	1,2,3,4,5	1,2,3,4,5

Insertion is $\Theta(n)$ best case on sorted input, $\Theta(n^2)$ worst case on reverse-sorted input, with $O(1)$ extra space. Using $>$ rather than $\geq$ in its shift condition keeps equal records in their original order, making it stable. It works especially well on small or nearly sorted inputs.

Check your answer. Selection's prefix is final. Insertion's prefix is sorted, but later insertions may move its values.

12 Merge sort: every split and merge

Source links: 2024 Q6(b); 2025 Q4(b).

One value is already sorted. Split the input until this is true everywhere; then combine sorted parts. During a merge, only the first unused value of each part can be the next smallest.

Algorithm: MERGE-SORT(A)

Step 0. Start.

Step 1. If $n \leq 1$, return A and go to End.

Step 2. Split A at $m = \lfloor n/2 \rfloor$. Recursively sort $A[0 : m]$ into L and $A[m : n]$ into R (half-open ranges).

Step 3. Return MERGE(L, R), using the steps below.

Step 4. End.

Algorithm: MERGE two sorted arrays L, R

Step 0. Start.

Step 1. Create output B; set $i = j = 0$.

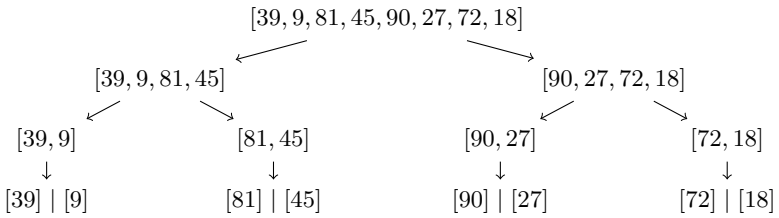
Step 2. While both arrays have unused items, compare $L[i]$ and $R[j]$. Append the smaller one and advance its index. On equality take from L for stability.

Step 3. Append all remaining items from whichever array is not exhausted. Return B.

Step 4. End.

Paper example: $39, 9, 81, 45, 90, 27, 72, 18$

Each row in the split diagram is a new recursion level. Vertical bars separate independent subproblems.



Merge level	Results of every merge
Singles to pairs	$[9, 39]$ $[45, 81]$ $[27, 90]$ $[18, 72]$
Pairs to fours	$[9, 39, 45, 81]$ $[18, 27, 72, 90]$
Fours to eight	$[9, 18, 27, 39, 45, 72, 81, 90]$

Final merge step	Compare current fronts	Take	Output so far
1	9 vs 18	9	9
2	39 vs 18	18	9,18
3	39 vs 27	27	9,18,27
4	39 vs 72	39	9,18,27,39
5	45 vs 72	45	9,18,27,39,45
6	81 vs 72	72	9,18,27,39,45,72
7	81 vs 90	81	9,18,27,39,45,72,81
8	Left exhausted	90	9,18,27,39,45,72,81,90

Why $\Theta(n \log n)$? Splitting has about $\log_2 n$ levels. Each merge level processes n values in total, so $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ in all standard cases. A reusable merge buffer needs $O(n)$ auxiliary space; recursion adds $O(\log n)$. Standard array merge sort is stable if ties take the left input first.

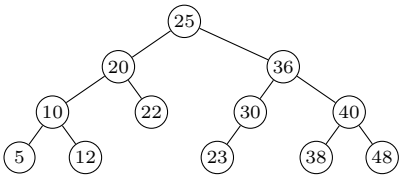
2025 variant: split $[45, 21, 67, 30, 19, 55, 20, 70]$ in the same way. Pair merges: $[21, 45], [30, 67], [19, 55], [20, 70]$; four-item merges: $[21, 30, 45, 67]$, $[19, 20, 55, 70]$; final: $[19, 20, 21, 30, 45, 55, 67, 70]$.

13 Tree language and the three traversals

Source links: 2023 Q4(b); 2024 Q1(c), Q4(b).

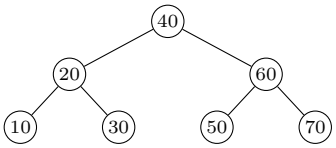
A tree is connected and has no cycle. In a rooted tree, every nonroot node has one parent. A binary tree has at most two ordered child positions: left and right. A child can be absent. A **leaf** has no children; an **internal node** has at least one.

Terminology: paper tree



Use the longest downward path to measure height.

Traversal practice tree



The same tree supports all three visit orders.

In the paper tree, node 20 has **degree 2** (two children); node 10's sibling is 22 (same parent). Root 25 has depth 0 and level 1. The deepest nodes have depth 3, so there are 4 levels. Node 36 has height 2 edges along $36 \to 40 \to 48$ or $36 \to 30 \to 23$. A leaf has height 0; the height of the whole tree is the root's height. State your convention if a class counts nodes instead of edges.

A **full** binary tree has 0 or 2 children at every node; a **complete** tree fills levels from left to right with only the last possibly incomplete; a **perfect** tree has all leaves at the same depth and every internal node has 2 children. A binary tree need not be a search tree: ordering is an extra rule.

Traversal names tell you when to visit the root

Algorithm: PREORDER(node): root, left, right

Step 0. Start.

Step 1. If node is NULL, return and go to End.

Step 2. Visit node; call PREORDER(node.left); call PREORDER(node.right); return.

Step 3. End.

Algorithm: INORDER(node): left, root, right

Step 0. Start.

Step 1. If node is NULL, return and go to End.

Step 2. Call INORDER(node.left); visit node; call INORDER(node.right); return.

Step 3. End.

Algorithm: POSTORDER(node): left, right, root

Step 0. Start.

Step 1. If node is NULL, return and go to End.

Step 2. Call POSTORDER(node.left); call POSTORDER(node.right); visit node; return.

Step 3. End.

For the seven-node practice tree, the exact visit at each step is:

Visit number	Preorder	Inorder	Postorder
1	40	10	10
2	20	20	30
3	10	30	20
4	30	40	50
5	60	50	70
6	50	60	60
7	70	70	40

Inorder of a BST is sorted, but inorder of an arbitrary binary tree need not be. Each traversal touches every node once: $\Theta(n)$ time; recursive auxiliary space $O(h+1)$, where h is height. A skewed tree can need $O(n)$ stack space.

Remember. PRE = parent before children; IN = parent in between; POST = parent after children. "Visit" means output the node at that point, not merely reach it.

14 BST insertion: every branch, every new node

Source link: 2023 Q4(a). Construction order: 60,25,72,15,30,68,101,13,18.

A binary search tree (BST) stores every smaller key somewhere in the left subtree and every larger key somewhere in the right subtree. This rule applies to *all descendants*, not just immediate children. Do not rearrange earlier keys to make the drawing balanced.

Algorithm: Insert a distinct key x into a BST

Step 0. Start.

Step 1. If root is NULL, create a node containing x, return it as root, and go to End.

Step 2. Start $p \leftarrow root$. Compare x with $p.key$. If equal, apply the stated duplicate policy and go to End.

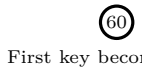
Step 3. If $x < p.key$, choose the left child; otherwise choose the right child.

Step 4. If that child exists, move p there and repeat Step 2. If it is empty, attach a new leaf containing x there. Return root.

Step 5. End.

Step 0: the tree is empty. In each snapshot the thicker circle is the newly inserted node. A caption such as $60L \rightarrow 25R$ means "left at 60, then right at 25." The final move always reaches an empty child link.

1. Insert 60



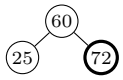
First key becomes root.

2. Insert 25



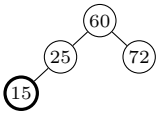
60L→ vacant link

3. Insert 72



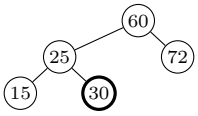
60R→ vacant link

4. Insert 15



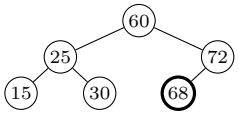
60L→25L→ vacant link

5. Insert 30



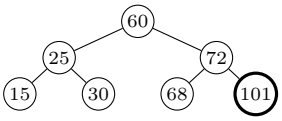
60L→25R→ vacant link

6. Insert 68



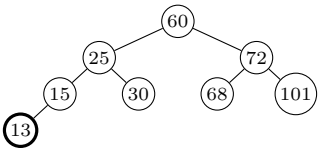
60R→72L→ vacant link

7. Insert 101



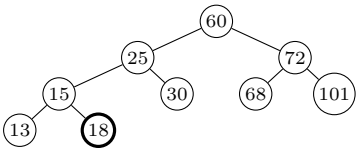
60R→72R→ vacant link

8. Insert 13



60L→25L→15L→ vacant link

9. Insert 18



60L→25L→15R→ vacant link

Why does 18 go there? $18 < 60$, so it must be in 60's left subtree; $18 < 25$, so go left again; $18 > 15$, so it must be 15's right child. This position satisfies every ancestor: $15 < 18 < 25 < 60$. Putting it below 30 would fail the comparison at 25.

Verify the finished tree. Inorder is $13, 15, 18, 25, 30, 60, 68, 72, 101$. Preorder is $60, 25, 15, 13, 18, 30, 72, 68, 101$. Postorder is $13, 18, 15, 30, 25, 68, 101, 72, 60$.

The root has 5 nodes in its left subtree and 3 in its right: $5+3+1 = 9$. Insertion and search follow one root-to-leaf path, costing $O(h+1)$: $O(\log n)$ if balanced, $O(n)$ if skewed. An ordinary BST does not rebalance itself.

15 A difficult BST: the first eight insertions

Source link: 2024 Q5(b). All 15 insertions are drawn across pages 15–16.

The full sequence is $13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18$. This question is useful because "smaller than the root" is not enough to locate a node. Every comparison on its route narrows the legal interval.

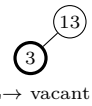
Step 0: root is empty. Apply the page 14 algorithm without rotating or sorting the input. Each picture is the entire tree after the indicated insertion; L and R refer to the comparison node in the caption.

1. Insert 13



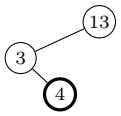
First key becomes root.

2. Insert 3



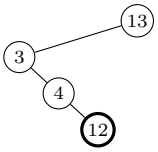
13L→ vacant link

3. Insert 4



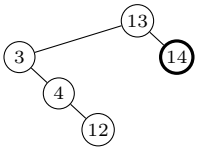
13L→3R→ vacant link

4. Insert 12



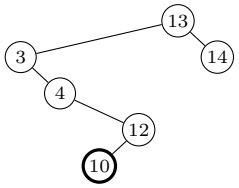
13L→3R→4R→ vacant link

5. Insert 14



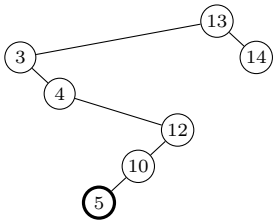
13R→ vacant link

6. Insert 10



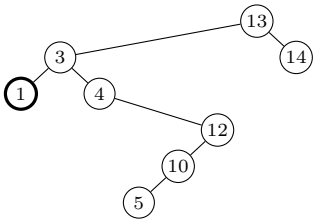
13L→3R→4R→12L→ vacant link

7. Insert 5



13L→3R→4R→12L→10L→ vacant link

8. Insert 1



13L→3L→ vacant link

Explain the route, rather than memorise the drawing

Key	Comparisons	Why this branch is forced
13	No root yet	First item is the root.
3	$3 < 13$	Left child of 13.
4	$4 < 13$, $4 > 3$	Right child of 3; it stays inside the interval $(3, 13)$.
12	$12 < 13$, $12 > 3$, $12 > 4$	Right child of 4; interval $(4, 13)$.
14	$14 > 13$	Right child of 13.
10	$10 < 13$, $10 > 3$, $10 > 4$, $10 < 12$	Left child of 12; interval $(4, 12)$.
5	$5 < 13$, $5 > 3$, $5 > 4$, $5 < 12$, $5 < 10$	Left child of 10; interval $(4, 10)$.
1	$1 < 13$, $1 < 3$	Left child of 3; the deep right branch of 3 is irrelevant.

How to draw cleanly in the exam. Keep the root in a fixed position, add only the new leaf, and write L/R decisions beside the working. You may lengthen a branch to make space; that does not change the tree. You may not move a key to a different parent just to fit the page.

Check your answer. After eight insertions, inorder is $1, 3, 4, 5, 10, 12, 13, 14$. Eight values entered, eight nodes appear. Key 5 lies to the right of 4 but to the left of 10; both ancestor constraints matter.

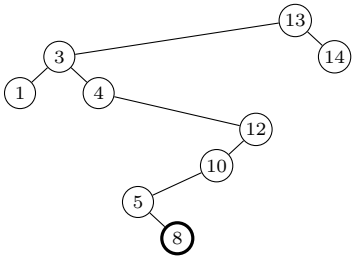
A useful error test. If 9 were placed to the right of 10, inorder would stop being sorted. It must be > 8 but < 10. If 6 were placed directly below 5, it would bypass the already present child 8. Always follow existing links before creating a new one.

16 The difficult BST: finish and verify

Source link: 2024 Q5(b), continued. These snapshots include all earlier nodes.

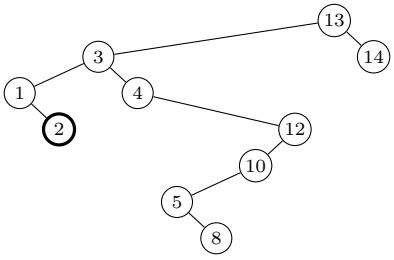
Continue from the eight-node state on page 15. Deep paths are normal here: an ordinary BST inherits the insertion order. An empty-looking space in the drawing is not a legal insertion slot unless the comparison path reaches its parent.

9. Insert 8



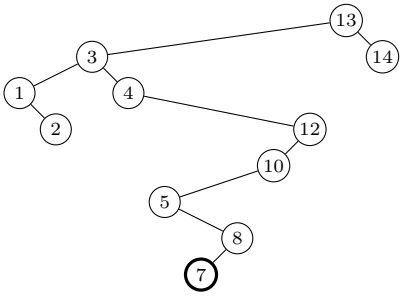
13L→3R→4R→12L→10L→5R→ vacant link

10. Insert 2



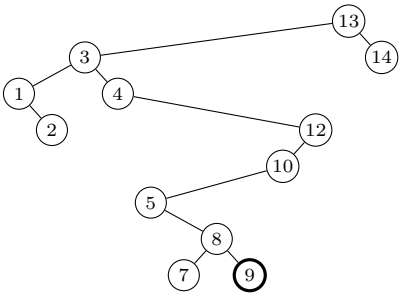
13L→3L→1R→ vacant link

11. Insert 7



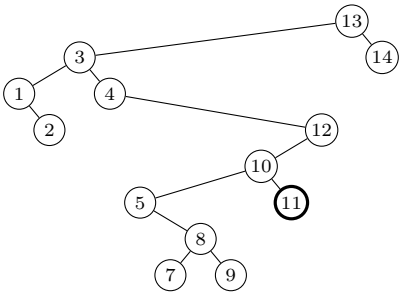
13L→3R→4R→12L→10L→5R→8L→ vacant link

12. Insert 9



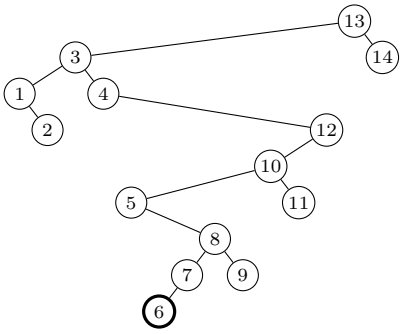
13L→3R→4R→12L→10L→5R→8R→ vacant link

13. Insert 11



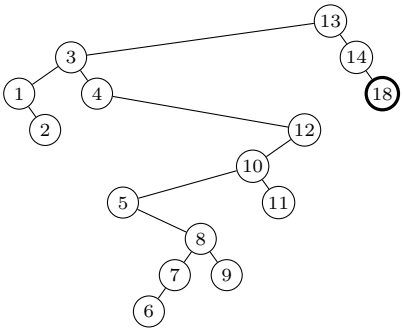
13L→3R→4R→12L→10R→ vacant link

14. Insert 6



13L→3R→4R→12L→10L→5R→8L→7L→ vacant link

15. Insert 18



13R→14R→ vacant link

New key	Route, including every comparison node	Final attachment
8	$13L \to 3R \to 4R \to 12L \to 10L \to 5R$	Right child of 5.
2	$13L \to 3L \to 1R$	Right child of 1.
7	$13L \to 3R \to 4R \to 12L \to 10L \to 5R \to 8L$	Left child of 8.
9	$13L \to 3R \to 4R \to 12L \to 10L \to 5R \to 8R$	Right child of 8.
11	$13L \to 3R \to 4R \to 12L \to 10R$	Right child of 10.
6	$13L \to 3R \to 4R \to 12L \to 10L \to 5R \to 8L \to 7L$	Left child of 7.
18	$13R \to 14R$	Right child of 14.

Longest path: $13 \to 3 \to 4 \to 12 \to 10 \to 5 \to 8 \to 7 \to 6$ has 8 edges. Thus height is 8, and finding 6 examines 9 nodes. This explains why assuming every BST operation costs $\log n$ loses marks.

Final inorder: $1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 18$. It matches the sorted input exactly. The root's left subtree has 12 nodes and the right has 2; $12 + 2 + 1 = 15$.

17 Duplicates and counting nodes correctly

Source links: 2025 Q1(c); subtree-count method also answers 2024 Q4(b)(i).

This sequence contains 23 twice: $98, 2, 48, 12, 56, 32, 4, 67, 23, 87, 23, 55, 46$. We store a **count** at a key's node. On equality, increment the count and stop; do not invent a new branch. An alternative classroom rule is "equal goes right"; if required, use it consistently and draw a separate equal-key node.

Step 0: empty tree. A node marked 23×2 stores two occurrences. The thicker outline identifies the key just inserted or updated.

1. Insert 98



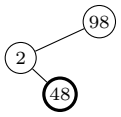
First key becomes root.

2. Insert 2



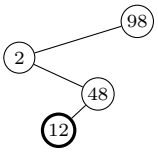
98L→ vacant link

3. Insert 48



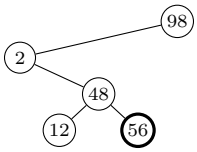
98L→2R→ vacant link

4. Insert 12



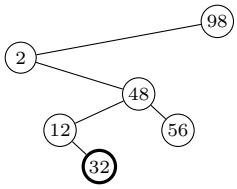
98L→2R→48L→ vacant link

5. Insert 56



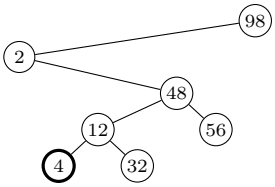
98L→2R→48R→ vacant link

6. Insert 32



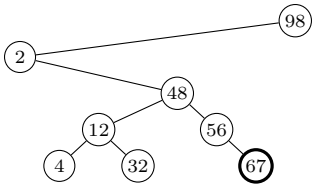
98L→2R→48L→12R→ vacant link

7. Insert 4



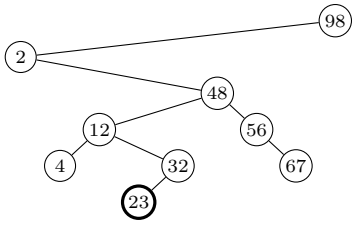
98L→2R→48L→12L→ vacant link

8. Insert 67



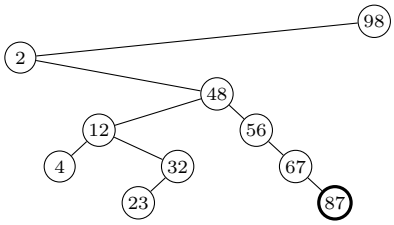
98L→2R→48R→56R→ vacant link

9. Insert 23



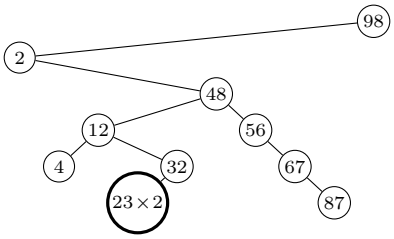
98L→2R→48L→12R→32L→ vacant link

10. Insert 87



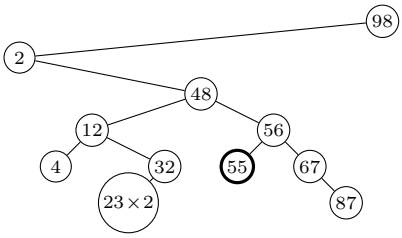
98L→2R→48R→56R→67R→ vacant link

11. Insert 23



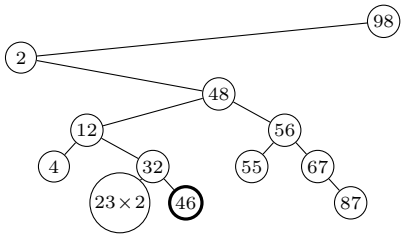
Equal key: increment count; no new branch.

12. Insert 55



98L→2R→48R→56L→ vacant link

13. Insert 46



98L→2R→48L→12R→32R→ vacant link

Check: there are 13 input occurrences but 12 physical nodes. Inorder by distinct node is $2, 4, 12, 23, 32, 46, 48, 55, 56, 67, 87, 98$. To list all stored occurrences, emit 23 twice. The root 98 has 11 physical nodes on its left and none on its right.

Subtree counts: count all descendants on that side, not merely immediate children. In the separate 2024 sequence $50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24$, the values below 50 are $15, 5, 20, 3, 8, 37, 24$ (7); those above are $62, 58, 91, 60$ (4). Thus $7 + 4 + 1 = 12$. The insertion method is unchanged; the count can be checked directly against the root.

18 Reconstruct a binary tree from traversals

Source links: 2023 Q4(b); the identical traversal pair repeats in 2024 Q4(b)(ii).

Given inorder: $4, 8, 2, 5, 1, 6, 3, 7$. **Given postorder:** $8, 4, 5, 2, 6, 7, 3, 1$.

Postorder ends with the root. Inorder tells which nodes belong on each side. Do not compare the numerical sizes here: this is a general binary-tree reconstruction, not BST insertion. For example, 2 will be left of 1.

Algorithm: BUILD(inorder I, postorder P), with distinct keys

Step 0. Start.

Step 1. If both sequences are empty, return NULL and go to End. Otherwise require matching lengths and the same keys.

Step 2. Take the last element of P as root r. Find r in I; let k be the number of items before it.

Step 3. The first k inorder items form I_L; the items after r form I_R. The first k postorder items form P_L; the remaining items before the final root form P_R.

Step 4. Create r; set its left child to BUILD(I_L, P_L) and its right child to BUILD(I_R, P_R). Return r.

Step 5. End.

Root being solved	Left inorder / postorder	Right inorder / postorder
1	$4, 8, 2, 5$ / $8, 4, 5, 2$	$6, 3, 7$ / $6, 7, 3$
2	$4, 8$ / $8, 4$	5 / 5
4	empty / empty	8 / 8
3	6 / 6	7 / 7

Step 0: empty construction. We build the left subtree fully, then the right; the following are all eight node attachments. Splitting into blocks first makes the branch decisions explicit.

1. Root 1



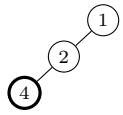
Last postorder item is 1.

2. Attach 2



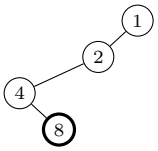
Left block ends in 2.

3. Attach 4



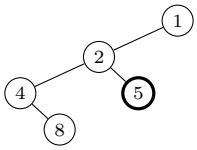
Left block of 2 ends in 4.

4. Attach 8



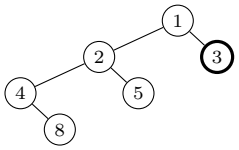
Inorder is 4,8: 8 is right of 4.

5. Attach 5



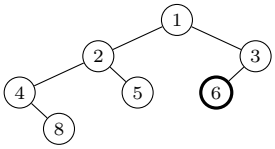
Right block of 2 is just 5.

6. Attach 3



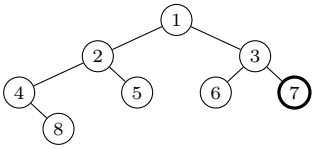
Right block of 1 ends in 3.

7. Attach 6



6 is left of 3 in inorder.

8. Attach 7



7 is right of 3 in inorder.

Why is 8 a right child? The subtree's inorder is $[4, 8]$ and its postorder ends in 4. Root is 4; there is nothing before 4 in inorder, and 8 is after it. Therefore left is empty and right is 8, even though another drawing might look more symmetric.

Check your answer. Read both supplied traversals back from the finished tree. Preorder is $1, 2, 4, 8, 5, 3, 6, 7$. With distinct keys, inorder plus preorder or postorder determines a unique binary tree. A direct slicing/search implementation can take $O(n^2)$ time; an index map and index ranges reduce it to $O(n)$.

19 BST reconstruction and deletion

Source links: 2025 Q5(b), Q3(b).

Reconstruct a BST from its preorder

Given preorder $40, 20, 10, 30, 60, 50, 70$ and postorder $10, 30, 20, 50, 70, 60, 40$, insert preorder values with the page 14 algorithm. Root is 40; every value below it is in the left subtree. The seven insertion states are:

1. Insert 40



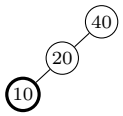
First key becomes root.

2. Insert 20



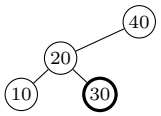
40L→ vacant link

3. Insert 10



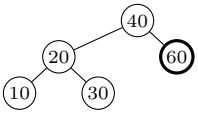
40L→20L→ vacant link

4. Insert 30



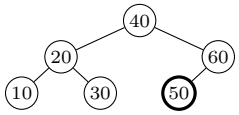
40L→20R→ vacant link

5. Insert 60



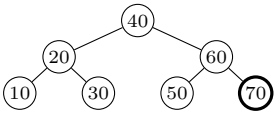
40R→ vacant link

6. Insert 50



40R→60L→ vacant link

7. Insert 70

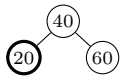


40R→60R→ vacant link

Inorder is $10, 20, 30, 40, 50, 60, 70$; postorder matches the given sequence. Distinct keys plus the BST ordering make this unique. Preorder and postorder alone do *not* uniquely identify every general binary tree: a single child could be left or right.

Deletion has exactly three structural cases

Leaf: before



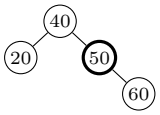
Delete 20: set its parent link to NULL.

Leaf: after



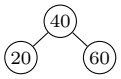
40 keeps its right child 60.

One child: before



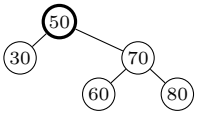
Delete 50; its only child is 60.

One child: after



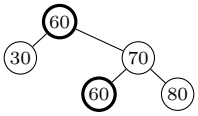
40.right points directly to 60.

Two children: before



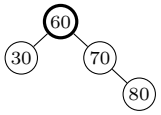
Delete 50; successor is 60.

Copy successor key



Temporary duplicate: old 60 still exists.

Two children: after



Delete old 60 from the right subtree.

Algorithm: DELETE(root, key), distinct-key BST

Step 0. Start.

Step 1. If root is NULL, return NULL and go to End (not found).

Step 2. If key is smaller, set root.left to DELETE(root.left,key) and go to Step 6. If larger, set root.right to DELETE(root.right,key) and go to Step 6.

Step 3. Key matches. If root.left is NULL, save root.right, release root and return the saved child; go to End.

Step 4. If root.right is NULL, save root.left, release root and return the saved child; go to End.

Step 5. Otherwise find successor s by going right once, then left until no left child exists. Copy $s.key$ to root; set root.right to DELETE(root.right,$s.key$).

Step 6. Return root. The caller assigns the returned pointer to its child link or to the overall root.

Step 7. End.

The successor is the smallest key larger than the deleted key; it cannot have a left child. Time is $O(h + 1)$. For counted duplicates, decrement a count above 1 first. If moving a successor that has a count, move the complete record and remove its old node completely, so the same distinct key does not occupy two nodes.

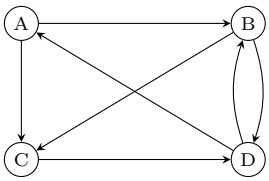
20 Graphs: edges, degree and representations

Source links: 2023 Q1(d), Q3(b), Q5(a); 2024 Q1(d).

A graph $G = (V, E)$ has vertices and edges. Undirected $u - v$ can be used either way; directed $u \to v$ can be used only forward. A weighted edge has a numerical cost. A path follows permitted edges. A cycle returns to its start without reusing intermediate vertices. A disconnected graph has vertices unreachable from some others.

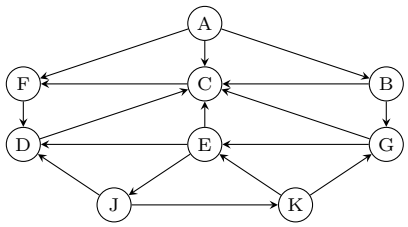
In an undirected graph, degree counts incident edges (a self-loop counts twice); total degree is $2|E|$. In a directed graph, **indegree** counts incoming arrows and **outdegree** outgoing arrows. Both total indegree and total outdegree equal $|E|$.

Four-vertex directed example



Read arrows from tail to arrowhead.

BFS graph used on page 21



This redraw preserves the source graph edges.

Adjacency matrix: row = from, column = to

For the four-vertex graph:

	A	B	C	D
A	0	1	1	0
B	0	0	1	1
C	0	0	0	1
D	1	1	0	0

Vertex	Outgoing adjacency list
A	B, C
B	C, D
C	D
D	A, B

There are 7 ones, hence 7 directed edges. For B, indegree is 2 (from A,D) and outdegree is 2 (to C,D). The matrix is not symmetric: A goes to B, but B does not go to A.

For an **unweighted** matrix, use 1 for an edge, 0 for none. For a **weighted distance** matrix, use the edge weight, ∞ for no edge, and 0 on the diagonal. Zero can be a valid edge weight, so it cannot always mean "no edge." Undirected matrices are symmetric.

Representation	Memory	Useful operations / trade-off
Adjacency matrix	$\Theta(V^2)$	Check a particular edge in $O(1)$; scan all neighbours in $O(V)$.
Adjacency list	$\Theta(V + E)$	Scan actual neighbours; good for sparse graphs. Undirected edges are stored at both endpoints.
Edge list	$\Theta(V + E)$ including vertices	Store triples (u, v, w); convenient for sorting edges in Kruskal.

Use the question's goal to choose the algorithm

BFS: reachability and fewest edges in an unweighted graph.

DFS: explore one branch deeply, then backtrack; useful for structural exploration.

Dijkstra: shortest weighted routes from one source, with nonnegative weights.

Prim / Kruskal: minimum total cost to connect all vertices of an undirected graph; routes from a chosen source are not the objective.

If a drawing repeats labels, distinguish the vertices explicitly (for example A-top, A-right). If the scan leaves a direction unclear, state your interpretation rather than silently changing an arrow. For any traversal, declare neighbour order before giving the sequence. **A second simple practice graph:** edges AB, AC, BD, BE, CF give DFS A, B, D, E, C, F. After D ends, return to B and explore E before going back to A. This is the backtracking rule; do not jump to C too early.

21 BFS: grow a discovery tree with a queue

Source link: 2023 Q5(b). The complete input graph is drawn on page 20.

A queue keeps earlier discoveries ahead of later ones. Mark a vertex *when enqueuing*, so two parents cannot enqueue the same vertex twice. Follow outgoing arrows, in alphabetical order. The drawings below show only discovery edges; other graph edges remain in the input but do not become parent links.

Algorithm: Breadth-first search from source s

Step 0. Start.

Step 1. Mark every vertex unvisited. Create an empty queue; mark s visited, set $d[s] = 0$, parent[s]=NULL, and enqueue s.

Step 2. If the queue is empty, go to End. Otherwise dequeue u and output u.

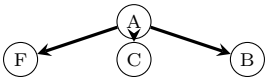
Step 3. For each outgoing neighbour v in the declared order, if unvisited: mark it, set parent[v]=u, $d[v] = d[u] + 1$, and enqueue v. Go to Step 2.

Step 4. End.

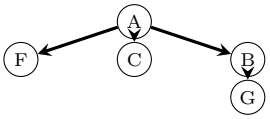
0. Queue A



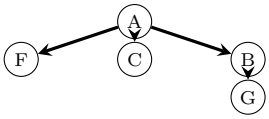
1. Remove A



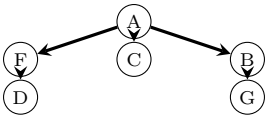
2. Remove B



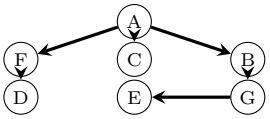
3. Remove C



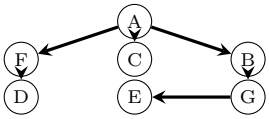
4. Remove F



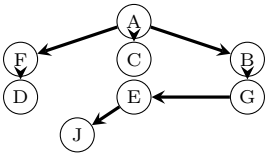
5. Remove G



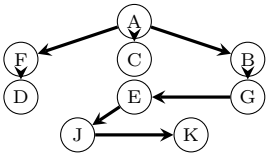
6. Remove D



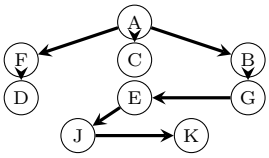
7. Remove E



8. Remove J



9. Remove K



Step	Removed	Queue after scanning neighbours	New discovery / reason
0	–	A	Start.
1	A	B,C,F	B,C,F are one edge away.
2	B	C,F,G	C already marked; add G.
3	C	F,G	F already marked; do not enqueue again.
4	F	G,D	Add D.
5	G	D,E	C visited; add E.
6	D	E	C visited.
7	E	J	C,D visited; add J.
8	J	K	D visited; add K.
9	K	empty	E,G visited; stop.

Order: $A, B, C, F, G, D, E, J, K$. Levels are $\{A\}$, $\{B, C, F\}$, $\{G, D\}$, $\{E\}$, $\{J\}$, $\{K\}$. The path $A \to B \to G \to E \to J \to K$ has five edges, the fewest possible to K. BFS uses $O(V + E)$ time with lists and $O(V)$ auxiliary space. To traverse a disconnected graph fully, restart from each remaining unvisited vertex.

22 DFS: go deep, then return to a choice

Source links: 2025 Q4(a); 2024 Q7(d); 2023 Q1(d).

DFS follows an unvisited neighbour immediately. A recursive call pauses its parent until the child finishes. The active call stack records the path to return along. Mark on entry, otherwise cycles cause repeated visits.

Algorithm: Depth-first search

Step 0. Start.

Step 1. Initially mark all vertices unvisited. Call VISIT(source).

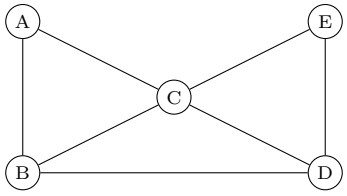
Step 2. VISIT(u): mark u visited and output u.

Step 3. For each neighbour v in the declared order, if v is unvisited, call VISIT(v) and wait for that call to finish before considering the next neighbour.

Step 4. When no neighbour remains, return to the caller. When the initial call returns, traversal from the source is complete.

Step 5. End.

Input, 2025 paper. Adjacency lists (undirected): A:B,C; B:A,C,D; C:A,B,D,E; D:B,C,E; E:C,D. Use alphabetical neighbour order.



Each discovery adds one tree edge below; the non-tree input edges are omitted from these discovery snapshots.

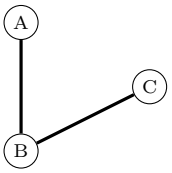
1. Enter A



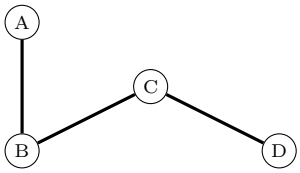
2. Enter B



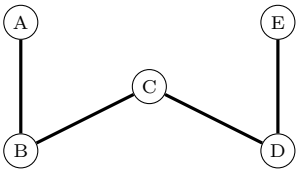
3. Enter C



4. Enter D



5. Enter E



Event	Active call stack after event	Reason
Enter A	A	B precedes C.
Enter B	A,B	A visited; C precedes D.
Enter C	A,B,C	A,B visited; choose D.
Enter D	A,B,C,D	B,C visited; choose E.
Enter E	A,B,C,D,E	C,D visited; no new neighbour.
Return from E	A,B,C,D	D has no remaining unvisited neighbour.
Return from D	A,B,C	E is already visited.
Return from C	A,B	D is already visited.
Return from B	A	C is already visited.
Return from A	empty	Traversal complete.

Discovery order: A, B, C, D, E. Returning from a node does not output it again. The final discovery tree is the chain $A - B - C - D - E$, although the input graph has additional edges.

With adjacency lists, DFS takes $O(V + E)$ time and $O(V)$ stack space; a matrix takes $O(V^2)$ time. DFS paths need not be shortest. The declared neighbour order fixes this trace.

23 Dijkstra: shortest routes from one source

Source link: 2024 Q3(a). Requires nonnegative edge weights.

A tentative distance $d[v]$ is the cheapest complete source-to-v route found so far, not the weight of the last edge alone. **Settling** a vertex means its shortest distance is now final. **Relaxing** $u \to v$ asks whether a route through u improves v:

$$\boxed{\text{candidate} = d[u] + w(u,v); \quad \text{if candidate} < d[v],\ d[v] \leftarrow \text{candidate},\ \mathit{parent}[v] \leftarrow u.}$$

Algorithm: Dijkstra from source s using a simple minimum scan

Step 0. Start.

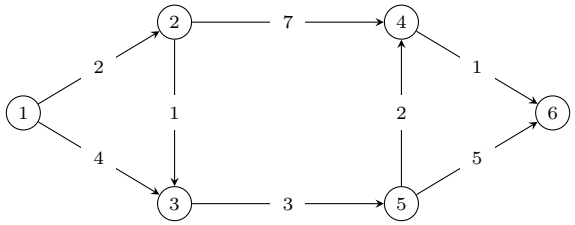
Step 1. Set every $d[v] \leftarrow \infty$, parent[v]←NULL, and every vertex unsettled. Set $d[s] \leftarrow 0$.

Step 2. Choose the unsettled vertex u with smallest finite $d[u]$ (numerical order breaks a tie). If none exists, go to End.

Step 3. Mark u settled. For each outgoing edge $u \to v$ to an unsettled vertex, relax it using the boxed rule.

Step 4. Repeat Step 2. Return distances and parents; vertices left at ∞ are unreachable.

Step 5. End.



The source is 1. Read only in arrow direction. Store columns in numerical vertex order even though the settle order differs.

After settling	d_1	d_2	d_3	d_4	d_5	d_6	Important update
Initial	0	∞	∞	∞	∞	∞	No route discovered yet.
1	0	2	4	∞	∞	∞	Parents of 2,3 become 1.
2	0	2	3	9	∞	∞	3 improves through 2; 4 discovered.
3	0	2	3	9	6	∞	5 discovered through 3.
5	0	2	3	8	6	11	4 improves; 6 discovered.
4	0	2	3	8	6	9	6 improves through 4.
6	0	2	3	8	6	9	No further update.

Every decision explained

Settle 1 (0): $0 + 2 < \infty$ gives $d_2 = 2$; $0 + 4 < \infty$ gives $d_3 = 4$. Choose 2 next because $2 < 4$.

Settle 2 (2): for 3, $2 + 1 = 3 < 4$, so replace its parent 1 with 2; for 4, $2 + 7 = 9$. Choose 3 next because $3 < 9$.

Settle 3 (3): $3 + 3 = 6$ reaches 5. Choose 5 next because $6 < 9$; the next vertex is not simply the next numerical label.

Settle 5 (6): $6 + 2 = 8 < 9$ improves 4; $6 + 5 = 11$ reaches 6. Choose 4 because $8 < 11$.

Settle 4 (8): $8 + 1 = 9 < 11$ improves 6. Settle 6 last. Its final distance is 9.

Why settling is safe: with nonnegative weights, a later route through a currently more distant unsettled vertex cannot lead back to a smaller distance for the chosen minimum. Negative edges break that reasoning. The simple implementation costs $O(V^2)$ time and $O(V)$ auxiliary space beyond the graph; a binary heap with adjacency lists gives $O((V + E) \log V)$ time.

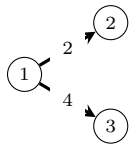
24 Dijkstra diagrams, routes and unreachable nodes

Source links: 2024 Q3(a), continued; 2023 Q3(b).

The predecessor tree changes whenever a route improves

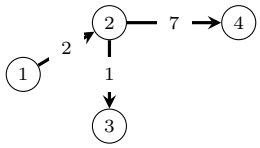
These are the parent links after every settling step of page 23. An unsettled vertex's parent is still tentative; it may move. Edge labels are original edge weights, not accumulated distances.

After 1



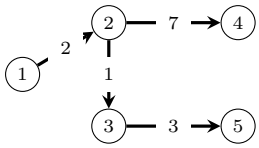
Tentative paths: 2 via 1, 3 via 1.

After 2



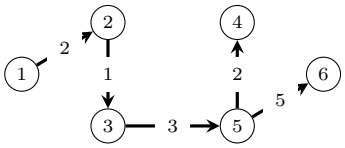
Replace the old parent of 3.

After 3



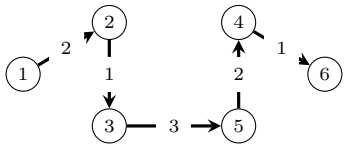
Add the route to 5, cost 6.

After 5



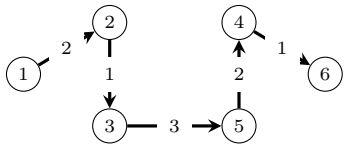
Replace parent of 4; reach 6 at 11.

After 4



Replace parent of 6; cost becomes 9.

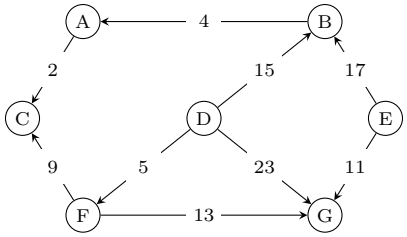
After 6



All shown distances are final.

To recover a route, follow parents backward and reverse it. For 6: $6 \leftarrow 4 \leftarrow 5 \leftarrow 3 \leftarrow 2 \leftarrow 1$. Therefore the route is $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$, costing $2 + 1 + 3 + 2 + 1 = 9$.

A second paper graph: direction makes E unreachable



Start at D. Its outgoing edges reach B,F,G, but no permitted route enters E. In a weighted adjacency matrix, the non-infinite off-diagonal entries are exactly the nine drawn arrows; all other off-diagonal entries are ∞.

After settling	A	B	C	D	E	F	G
Initial	∞	∞	∞	0	∞	∞	∞
D	∞	15	∞	0	∞	5	23
F	∞	15	14	0	∞	5	18
C	∞	15	14	0	∞	5	18
B	19	15	14	0	∞	5	18
G	19	15	14	0	∞	5	18
A	19	15	14	0	∞	5	18

Updates: F gives $d_C = 5 + 9 = 14$ and improves d_G from 23 to $5 + 13 = 18$. B gives $d_A = 15 + 4 = 19$; A's candidate for C is $19 + 2 = 21$, worse than 14. Shortest routes are D–F (5), D–F–C (14), D–B (15), D–F–G (18), D–B–A (19); E has no route. D itself has distance 0.

Check your answer. Along every reachable edge, final values satisfy $d[v] \leq d[u] + w(u, v)$. Sum each reported path's weights. Never reverse E→B to manufacture B→E. Stop when no finite unsettled distance remains.

Remember. Dijkstra chooses minimum source distance; Prim chooses a minimum crossing edge; Kruskal chooses the next global edge that joins two components.

25 Prim: grow one minimum spanning tree

Source links: 2025 Q6(a); 2024 Q5(a).

A spanning tree connects all V vertices without a cycle; it has $V - 1$ edges. A minimum spanning tree (MST) minimises the sum of those edge weights. It models a cheapest cable or pipeline backbone. Prim grows one connected set S by choosing the cheapest edge from *inside* to *outside* S. It does not add accumulated path distances.

Algorithm: Prim on a connected, weighted, undirected graph

Step 0. Start.

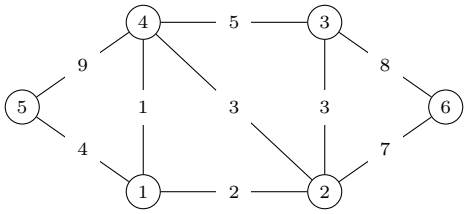
Step 1. Set $S \leftarrow \{start\}$, accepted edges $T \leftarrow \varnothing$, total $\leftarrow 0$.

Step 2. If all vertices are in S, return T and total, and go to End.

Step 3. Find a minimum-weight edge with exactly one endpoint in S. If none exists, report that no spanning tree covers the entire graph and go to End.

Step 4. Add the edge to T, its outside endpoint to S, and its weight to total. Go to Step 2.

Step 5. End.

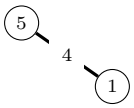


0. Start at 5



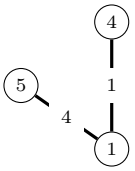
Cost 0; no edge yet.

1. Add 5–1



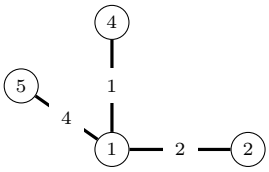
New vertex 1; edge cost 4.

2. Add 1–4



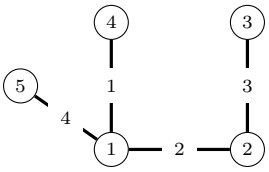
New vertex 4; edge cost 1.

3. Add 1–2



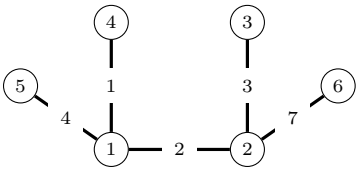
New vertex 2; edge cost 2.

4. Add 2–3



New vertex 3; edge cost 3.

5. Add 2–6



New vertex 6; edge cost 7.

Step	S before choosing	Crossing candidates (edge:weight)	Chosen / running total
1	5	5–1:4, 5–4:9	5–1 / 4
2	5,1	1–4:1, 1–2:2, 5–4:9	1–4 / 5
3	5,1,4	1–2:2, 4–2:3, 4–3:5	1–2 / 7
4	5,1,4,2	2–3:3, 4–3:5, 2–6:7	2–3 / 10
5	5,1,4,2,3	2–6:7, 3–6:8	2–6 / 17

The final MST has 5 edges and cost $4 + 1 + 2 + 3 + 7 = \boxed{17}$. At step 1, edge 1–4 of weight 1 is ineligible because neither endpoint is in $S = \{5\}$. At step 3, edge 1–4 is no longer eligible because both endpoints are already included.

2024 variant from A: input edges AB:2, AC:2, BC:3, BD:6, BE:2, CD:5, CF:3, DE:4, DF:1. One valid order is AB, AC, BE, CF, FD; cost $2 + 2 + 2 + 3 + 1 = 10$. FD of weight 1 becomes eligible only after F enters. Ties can change the order or the MST, so state your tie rule.

Why it works: a cheapest edge across the current inside/outside cut can safely belong to an MST. The simple matrix implementation takes $O(V^2)$; a heap/list version $O((V + E) \log V)$. Negative edge weights are allowed for MSTs.

26 Kruskal: join components without a cycle

Source link: 2023 Q5(a). Use the same MST goal as Prim, but a different selection rule.

Kruskal begins with every vertex isolated. It sorts *all* edges by weight, then accepts an edge only when its endpoints are in different connected components. Unlike Prim, its intermediate accepted edges can be disconnected.

Algorithm: Kruskal minimum spanning tree

Step 0. Start.

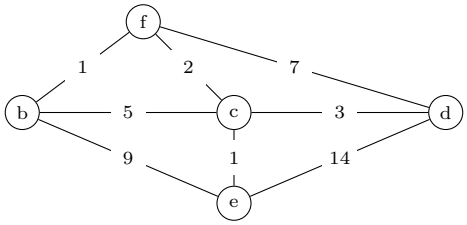
Step 1. Set $T \leftarrow \varnothing$, total $\leftarrow 0$. Put every vertex into its own component.

Step 2. Sort edges in nondecreasing weight order; use a declared order for ties.

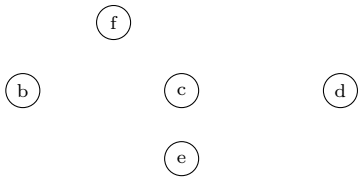
Step 3. Inspect the next edge (u, v, w). If u, v are in different components, accept it, merge the two components, and add w to total; otherwise reject it as a cycle.

Step 4. Stop successfully after $V - 1$ accepted edges. Otherwise repeat Step 3 while edges remain. If they run out first, report a minimum spanning forest and no single spanning tree.

Step 5. End.

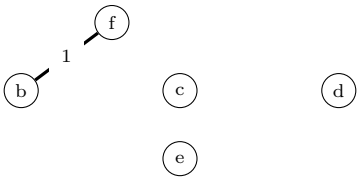


0. Five components



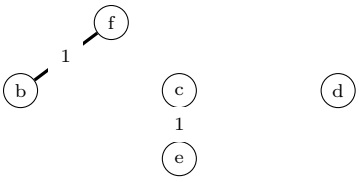
Each vertex begins alone.

1. Accept b–f



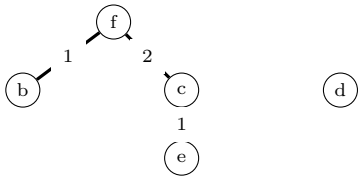
Components remaining: 4.

2. Accept c–e



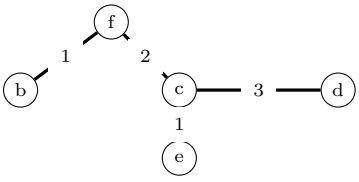
Components remaining: 3.

3. Accept f–c



Components remaining: 2.

4. Accept c–d



Components remaining: 1.

Sorted edge	Weight	Endpoints connected already?	Decision / components
b–f	1	No	Accept: $\{b, f\}, \{c\}, \{d\}, \{e\}$.
c–e	1	No	Accept: $\{b, f\}, \{c, e\}, \{d\}$.
f–c	2	No	Accept: $\{b, c, e, f\}, \{d\}$.
c–d	3	No	Accept: all connected; stop.
b–c	5	Yes (if inspected)	Reject: b–f–c is already a path.
f–d	7	Yes (if inspected)	Reject cycle.
b–e	9	Yes (if inspected)	Reject cycle.
d–e	14	Yes (if inspected)	Reject cycle.

Result: $T = \{bf, ce, fc, cd\}$, cost $1 + 1 + 2 + 3 = \boxed{7}$. There are 5 vertices and 4 accepted edges, no cycle and one component. On page 25's graph, Kruskal would reject 4–2 (weight 3) after accepting 4–1 and 1–2, even while other vertices remain isolated. Cycle rejection can happen before completion too.

A disjoint-set structure supports **FIND** (which component?) and **UNION** (merge components). Sorting dominates the usual running time: $O(E \log E)$; graph storage is $O(V + E)$. The same cut-safety principle justifies accepted edges: an edge connecting separate current components is safe when it is the next cheapest available one.

27 Huffman coding: merge the two smallest

Source link: 2025 Q7(b). Every merge is shown as a tree, not just a list of weights.

A fixed-length code for six symbols needs $\lceil \log_2 6 \rceil = 3$ bits per symbol. Huffman gives frequent symbols short codes and rare ones longer codes. Codes must be **prefix-free**: no complete symbol code is the beginning of another. This lets a decoder identify each symbol without separators.

Input frequencies are $P:30, Q:3, R:6, S:12, T:34, U:7$ (total 92). A frequency is a count of occurrences. A parent weight is the sum of all occurrences in its subtree. An internal node represents a *group*, not a new character.

Algorithm: Huffman tree and codebook

Step 0. Start.

Step 1. Create a leaf for each symbol with positive frequency and put all leaves in a min-priority queue.

Step 2. If more than one root remains, extract the two smallest weights x, y. Create parent z with weight $x.weight + y.weight$, left child x, right child y.

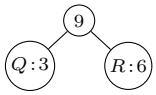
Step 3. Put z back into the queue; repeat Step 2 until exactly one root remains.

Step 4. Label each left edge 0 and right edge 1. For each leaf, concatenate root-to-leaf bits to obtain its symbol code. Return root and codebook.

Step 5. End.

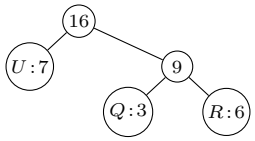
Step	Two roots removed	New root	Queue weights after reinsertion
0	–	–	3,6,7,12,30,34
1	Q:3, R:6	9	7,9,12,30,34
2	U:7, group:9	16	12,16,30,34
3	S:12, group:16	28	28,30,34
4	group:28, P:30	58	34,58
5	T:34, group:58	92	92; finished

1. Make 9



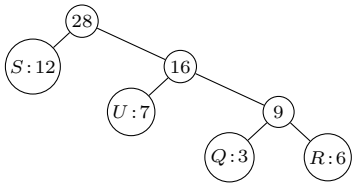
Leaves Q and R become siblings.

2. Make 16



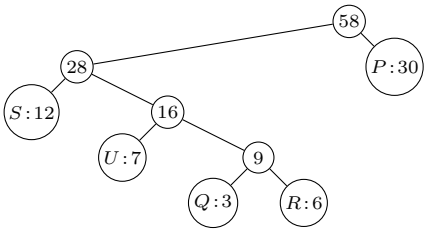
7 is smaller than 9; merge both.

3. Make 28



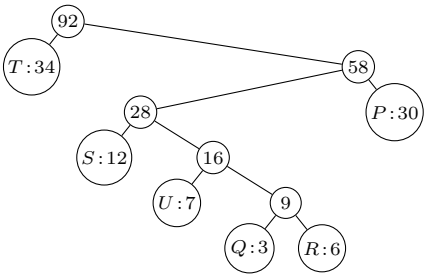
Merge 12 and 16.

4. Make 58



28 is smaller than 30 and 34.

5. Make 92



Merge the final two roots: 34 and 58.

Why must the new group return to the priority queue?

After the first merge, the available weights are 7,9,12,30,34. The next pair is 7 and 9, not the next two original letters. Ignoring the newly formed parent is the most common construction error. At every merge, re-sort or reselect the two smallest available roots.

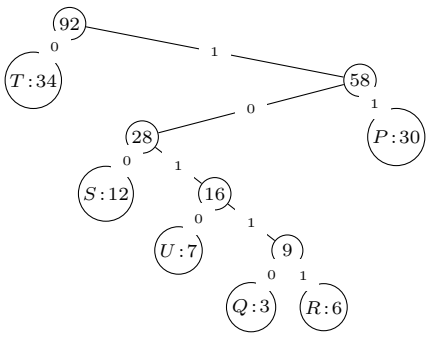
Why the greedy idea works: in an optimal binary prefix tree, two least frequent symbols can be placed as deepest siblings without increasing cost. Replace them by one combined-weight symbol; solving that smaller problem and expanding the pair recovers an optimal tree. Repeating this reduction gives Huffman's algorithm.

With a binary heap, k symbols need $O(k \log k)$ construction time and $O(k)$ tree space. Equal weights can lead to different equally optimal trees. With a single distinct symbol, agree on a convention such as code 0; the multi-symbol examples here need no special handling.

28 Huffman codes, decoding and a second tree

Source links: 2025 Q7(b), continued; 2023 Q7(b).

Read bits from the root of the 92-frequency tree



Symbol	Frequency	Root-to-leaf bits	Length	Frequency × length
T	34	0	1	34
P	30	11	2	60
S	12	100	3	36
U	7	1010	4	28
Q	3	10110	5	15
R	6	10111	5	30
Total	92	–	–	203 bits

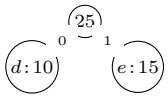
Fixed-length data uses $92 \times 3 = 276$ bits; Huffman uses 203, saving 73 bits for the encoded symbols. Codebook/header overhead is excluded. Average length is $203/92 \approx 2.207$ bits per symbol. As a second check, add all merge weights: $9 + 16 + 28 + 58 + 92 = 203$.

Decode step by step: for bitstream `011100`, start at root; 0 reaches T, so output T and reset; 11 reaches P, output P and reset; 100 reaches S, output S. Decoded text is TPS. Never continue downward after reaching a leaf. A stream ending inside the tree is incomplete.

Second paper example: $a : 30, b : 25, c : 20, d : 10, e : 15$

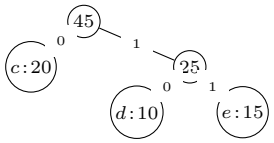
At the second merge, weight 25 is tied: choose the existing group (d, e) rather than b. State this tie choice. The four complete merges are:

1. $10 + 15 = 25$



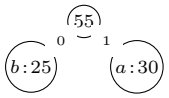
Queue: 20,25(b),25(group),30.

2. $20 + 25 = 45$



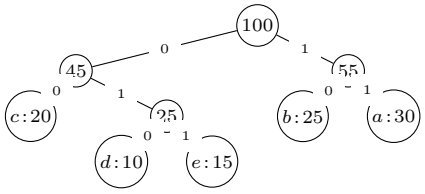
Merge c with the 25-group.

3. $25 + 30 = 55$



Merge b and a.

4. $45 + 55 = 100$



All original symbols now lie below the root.

Codes are $c = 00, d = 010, e = 011, b = 10, a = 11$. Weighted length is

$$20(2) + 10(3) + 15(3) + 25(2) + 30(2) = \boxed{225 \text{ bits}},$$

which also equals $25 + 45 + 55 + 100 = 225$. Another tie choice can change the bit strings while retaining the same optimum length. Swapping left and right also changes codes, not cost.

Check your answer. Every original symbol appears exactly once as a leaf. Every parent equals its child-weight sum. Root equals total frequency. Prefix-freeness follows because symbols live only at leaves.

29 Hashing and linear probing

Source links: 2023 Q7(c); 2024 Q7(a); 2025 Q1(d).

A hash function maps a key to a table index. For table size m, the common example is $h(k) = k \bmod m$. A **collision** means two keys want the same slot. Linear probing tries $h(k), h(k)+1, \ldots$, wrapping modulo m, until a usable slot appears.

Algorithm: Linear probing: INSERT or SEARCH (no deleted slots yet)

Step 0. Start.

Step 1. Compute $h \leftarrow k \bmod m$, or use the given hash function.

Step 2. For $i = 0, 1, \ldots, m-1$, inspect $j = (h+i) \bmod m$.

Step 3. If the slot already holds k, return found for SEARCH; apply the declared update policy for INSERT; go to End.

Step 4. If the slot is empty, return not found for SEARCH, or store k there for INSERT; go to End.

Step 5. Otherwise continue the loop. After all m probes, return not found for SEARCH or report full for INSERT.

Step 6. End.

In the 2023 question the hash function is omitted; declare $h(k) = k \bmod 10$. Insert keys in the *given* order, because collision destinations depend on earlier insertions.

Key	Home index	Every index tried	Placed at
27	7	7 empty	7
33	3	3 empty	3
64	4	4 empty	4
43	3	3 holds 33; 4 holds 64; 5 empty	5
37	7	7 holds 27; 8 empty	8
29	9	9 empty	9
30	0	0 empty	0
102	2	2 empty	2

Index	0	1	2	3	4	5	6	7	8	9
Key	30	–	102	33	64	43	–	27	37	29

Load factor $\alpha = n/m = 8/10 = 0.8$. More occupied cells generally mean longer probe sequences. A hash table gives expected $O(1)$ operations under good hashing and a controlled load; the worst case is $O(n)$, not guaranteed constant time.

A small table that makes collisions obvious

For $m = 5$ and keys 50,70,76,93:

Key	Home	Probe sequence	State after insertion, slots 0–4
50	0	0	50, –, –, –, –
70	0	0,1	50, 70, –, –, –
76	1	1,2	50, 70, 76, –, –
93	3	3	50, 70, 76, 93, –

Add practice key 98: $98 \bmod 5 = 3$, so try 3 then 4 and place it at 4. A further new key must inspect at most five slots and report full. For a wraparound example, if home 4 is occupied and slot 0 is empty, the next probe is 0, not 5.

Deletion warning: do not replace a removed key with an ordinary never-used empty marker. In the $m = 5$ example, clearing slot 1 after removing 70 would make a search for 76 stop too early. Use a *deleted/tombstone* marker and continue searches through it; insertion may reuse it after checking the key's probe chain for an existing key.

Disadvantages to write: primary clustering (long adjacent occupied runs), deteriorating performance near full capacity, deletion complexity, and a need to resize/rehash. Separate chaining is another collision strategy: a bucket stores a list of entries; it is not linear probing.

30 Closed-book recall and examination execution

Use the last 45 minutes to produce answers, rather than reread them.

Ten checks: cover the answers below first

1. Convert `A B C * + D -` to fully parenthesised infix. Which operand is popped first at the final minus?
2. Convert $A \wedge B \wedge C$ to postfix. Why not `A B ^ C ^`?
3. Insert 25 after node 20 in $10 \to 20 \to 30$. Write the two next-pointer assignments in order.
4. A circular queue has capacity 5, front 2, rear 1. Is it full? After removing the last item from any queue, what must the sentinel indices become?
5. In insertion sort, what is the first-pass result for $[4, 3, 1, 5, 2]$? What is selection sort's first-pass result?
6. Insert $40, 20, 60, 30, 25$ into a BST. Give the full route for 25 and the inorder check.
7. Reconstruct a tree from inorder B, A, C and postorder B, C, A. What are its root and children?
8. On page 23, when settling 5, why does d_4 become 8? Why is 4 chosen before 6 afterward?
9. Does a 6-vertex MST use six edges? How do Prim and Kruskal differ in the edge they consider next?
10. In Huffman coding, frequencies $2, 3, 7$ give which merge weights? With $m = 5$, where does 15 go if slots 0 and 1 are occupied but 2 is empty?

Answers. (1) $((A + (B * C)) - D)$; pop right operand D first. (2) `A B C ^ ^`; power associates right. (3) $q.next \leftarrow 20.next$, then $20.next \leftarrow q$. (4) Full because $(1 + 1) \bmod 5 = 2$; reset to $-1, -1$. (5) Insertion $[3, 4, 1, 5, 2]$; selection $[1, 3, 4, 5, 2]$. (6) $40L \to 20R \to 30L$; inorder $20, 25, 30, 40, 60$. (7) Root A, left B, right C. (8) $d_5 + w(5, 4) = 6 + 2 = 8 < 9$; 8 is below 6's tentative 11. (9) Five edges; Prim takes cheapest crossing edge, Kruskal next global non-cycle edge. (10) Merge 2+3=5, then 5+7=12; key 15 probes 0,1,2 and lands at 2.

Coverage map back to the supplied papers

F23 = 2023 final; R24 = 2024 re-exam; F25 = 2025 final. The map covers every concept family; repeated full questions are consolidated.

Concept / guide pages	Past-paper locations
Structure choice; costs; recursion (2–3)	F23 Q1(a),Q2(a),Q6(b); R24 Q1(a),Q2(a); F25 Q1(d),Q2(a)
Linked lists and memory diagrams (4–5)	F23 Q1(c),Q6(a); R24 Q1(b),Q4(a),Q6(a); F25 Q3(a),Q6(b)
Stacks and brackets (5–6)	R24 Q4(a); F25 Q1(a),Q2(b)
Expression conversion / trees (7–9)	F23 Q1(b),Q7(a); R24 Q2(a),Q3(b),Q7(a); F25 Q5(a)
Queues (10)	F23 Q2(b); R24 Q2(b),Q7(b); F25 Q7(a)
Selection, insertion, merge (11–12)	F23 Q3(a); R24 Q6(b),Q7(c); F25 Q1(b),Q4(b)
Tree terms, BST, traversals (13–19)	F23 Q4; R24 Q1(c),Q4(b),Q5(b); F25 Q1(c),Q3(b),Q5(b)
Graph representation; BFS/DFS (20–22)	F23 Q1(d),Q3(b),Q5; R24 Q1(d),Q7(d); F25 Q4(a)
Dijkstra; Prim; Kruskal (23–26)	F23 Q3(b),Q5(a); R24 Q3(a),Q5(a); F25 Q6(a)
Huffman; hashing (27–29)	F23 Q7(b–c); R24 Q7(a); F25 Q1(d),Q7(b)

An answer format that earns visible method marks

Write **definition / assumption → numbered algorithm → labelled trace or diagram → final answer → complexity and check**. Use intermediate states even when the final value is obvious. For a graph, state direction and tie order; for a BST, state duplicate policy; for expressions, preserve operand order.

The source papers are 100 marks in 180 minutes, with Q1 compulsory and four other questions. If your current paper uses the same format, a practical budget is 8 minutes reading, 30 for Q1, 33 for each chosen question and 10 for review. Read your actual paper's instructions first. Before stopping, check missing subparts, arithmetic, labels, empty/full tests and whether each algorithm ends.

Source note. Adapted from the user-supplied *DSA_PYQ_Complete_Annotated_Solutions.pdf*, 56 PDF pages (front matter plus printed pp. 1–52). Tree and graph diagrams are redrawn as vector illustrations; teaching checks and postfix-to-infix practice are added. The R24 expression with adjacent $C(D/E \wedge F)$ assumes multiplication: $A + (B * C * (D/E \wedge F) * G) * H$, giving `A B C * D E F ^ / * G * H * +`.